Text Analysis III

Sentiment Analysis and Text Classification

POLI3148 Data Science in Politics and Public Administration

Dr. Chen Haohan
The University of Hong Kong

Today's Question

How do we detect sentiments in documents at scale?

- **Part A:** Dictionary-based sentiment analysis
- **Part B:** Machine Learning Classifier for sentiment analysis

Part A

Dictionary-Based Sentiment Analysis

What is Sentiment Analysis?

- Assigns **positivity/negativity scores** to text
- Applications in political science:
  - Measuring diplomatic tone (our case of interest)
  - Public opinion and elite pronouncement on social media
  - Media bias detection
  - Legislative debate

How Dictionary Methods Work

- Match words against a pre-built **sentiment lexicon**
- Each word has a score (e.g., "excellent" = +3, "terrible" = -3)
- Aggregate scores across the document
- **VADER** (Valence Aware Dictionary and sEntiment Reasoner) also accounts for:
  - Punctuation ("Good!!!" scores higher than "Good")
  - Capitalization ("TERRIBLE" scores higher than "terrible")
  - Negation ("not good" flips the valence)
  - Degree modifiers ("very good" > "good")

Source: Hutto & Gilbert (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. *ICWSM*.

VADER Example

Text	Compound Score
"China welcomes this initiative and looks forward to cooperation"	+0.40 (positive)
"China strongly condemns this gross interference in internal affairs"	-0.65 (negative)
"The spokesperson held a regular press conference on Tuesday"	0.00 (neutral)

Compound score ranges from -1 (most negative) to +1 (most positive).

Applying VADER to MoFA Data

- Get sentiment scores for all questions and answers
- *Compare the score with the dataset's pre-computed* `q_sentiment` *column*
- Analysis: Compare sentiment by
  - Speakers
  - Time
  - News agents
  - Geographic entities of interest

Strengths and Weaknesses

Strengths: transparent, fast, reproducible, no training data needed

Weaknesses:

- Context-blind: "This is not bad" may be scored as negative
- Sarcasm: "What a wonderful decision" (sarcastic) scored as positive
- Domain-specific: diplomatic language has unique conventions
- No learning: the dictionary is fixed

Other Dictionary-Based Sentiment Tools

Lexicon	What It Measures	Reference
VADER	Positive / negative / neutral (with intensity)	Hutto & Gilbert 2014
Bing Liu	Binary positive / negative word list	Hu & Liu 2004
NRC Emotion	8 emotions (anger, fear, joy, ...) + positive/negative	Mohammad & Turney 2013
AFINN	Integer scores (-5 to +5) per word	Nielsen 2011
TextBlob	Polarity (-1 to +1) and subjectivity (0 to 1)	textblob.readthedocs.io

Different lexicons suit different tasks. VADER is popular for social media and short text; NRC is useful when you care about specific emotions.

Part B

Machine Learning Text Classification

When Dictionaries Aren't Enough

- We want to classify questions as "aggressive" or not
- This requires understanding **context**, not just individual words
- Solution: train a model that **learns patterns from labeled examples**

Machine Learning Classifier for Sentiment Analysis

- Instead of giving the machine a dictionary of sentiment words, give it examples of positive and negative questions/answers
- Machine *learn* patterns from the examples
- Then, machine apply the learned patterns to classify sentiments at scale

Key Concept: Machine Learning Classifier

A **classifier** is a model that predicts which **class** (category) a data point belongs to.

- You provide **labeled examples** (training data)
- The model learns patterns that distinguish the classes
- It then predicts labels for **new, unseen data**

Examples in political science:

Task	Classes
Is this press question aggressive?	Aggressive / Not aggressive
What is the topic of this speech?	Economy / Security / Health / ...
Does this document discuss human rights?	Yes / No
What is the sentiment of this tweet?	Positive / Negative / Neutral

Further reading: Grimmer, Roberts & Stewart (2022). *Text as Data*, Ch. 17--20. Cambridge

Building a Machine Learning Classifier

Data Input:

- Load text data
- Define **X** (text features, e.g., a document-term matrix) and **y** (target, e.g., sentiment type)
- Split the data into two subsets: training and test sets

Processing: Define model → Train model

Output: Evaluate model → Interpret model

Define X and y

- **y** (target): `q_aggressive` -- is this question aggressive?
  - Defined as bottom 25th percentile of sentiment scores
  - Normally obtained through manual labeling
- **X** (features): Bag-of-Words representation of question text
  - The document-term matrix from Session 2!

Train/Test Split

- We want to know: how well does the model perform on **new** data?
- **Split:** 80% training, 20% testing
- Train on training data, evaluate on test data
- Without splitting → **overfitting** (model memorizes instead of learning)

```
Full Dataset (N = 35,346)

┌──────────────────────────────────────────┬──────────┐
│       Training Set (80%)                 │Test (20%)│
│       28,276 rows                        │7,070 rows│
│                                          │          │
│  Model learns        Model NEVER         │  Used to │
│  patterns here       sees this  ─────    │ evaluate │
└──────────────────────────────────────────┴──────────┘
```

Training a Classifier

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
```

Two lines of code to train a text classifier. scikit-learn makes this simple.

Confusion Matrix

	Predicted: Not Aggressive	Predicted: Aggressive
Actual: Not Aggressive	True Negative ✓	False Positive ✗
Actual: Aggressive	False Negative ✗	True Positive ✓

The model makes **two types of errors**.

Evaluation Metrics

- **Accuracy:** (TP + TN) / Total -- what % correct overall?
- **Precision:** TP / (TP + FP) -- of predicted aggressive, how many actually are?
- **Recall:** TP / (TP + FN) -- of truly aggressive, how many did we catch?
- **F1 Score:** harmonic mean of precision and recall

Different applications care about different metrics.

Interpreting the Model

- Logistic regression: each word has a **coefficient**
- Largest positive coefficients = words predicting "aggressive"
  - e.g., "condemn", "attack", "accuse", "threaten"
- Largest negative coefficients = words predicting "not aggressive"
  - e.g., "congratulate", "welcome", "cooperation"
- This is **interpretable** -- we can explain each prediction

Try Multiple Models

- **Logistic Regression:** interpretable, good baseline
- **Random Forest:** often higher accuracy, less interpretable
- **Best practice:** try multiple models, compare performance
- scikit-learn: same workflow, just swap the model class

Classification Models in scikit-learn

All models follow the same API: `model.fit(X, y)` → `model.predict(X)`

Model	sklearn Command	Interpretability
Logistic Regression	`LogisticRegression()`	High -- word coefficients
Decision Tree	`DecisionTreeClassifier()`	High -- visualize the tree
Random Forest	`RandomForestClassifier()`	Medium -- feature importances
Gradient Boosting	`GradientBoostingClassifier()`	Medium -- feature importances
Support Vector Machine	`SVC()`	Low
Naive Bayes	`MultinomialNB()`	Medium -- word probabilities
k-Nearest Neighbors	`KNeighborsClassifier()`	Low
Neural Network (MLP)	`MLPClassifier()`	Low

Start with Logistic Regression (interpretable baseline), then experiment with others.

Preview: Large Language Models

What Traditional Text Analysis Approaches Required

1. **Manual preprocessing**: tokenization, BoW construction
2. **Feature engineering**: choosing which features to extract
3. **Labeled training data**: annotated examples to learn from (by yourself or others)

Each step requires human effort and domain expertise.

What Large Language Models Will Offer

- No feature engineering: the model processes **raw text**
- No or minimal labeled training data: just describe the task in **natural language**
- "Is this question aggressive? Answer yes or no."

But: ML classifiers are fast, transparent, reproducible, and free to run. There's no single best approach.

Hands-On

Notebook Demo

What We'll Do

- Apply VADER to MoFA questions
- Compare with dataset benchmarks
- Visualize sentiment trends over time and by spokesperson
- Train a logistic regression text classifier
- Evaluate with confusion matrix and metrics
- Interpret: which words predict aggressive questions?

Exercise

1. Apply VADER sentiment to the `answer` column
2. Do spokespeople respond more negatively to aggressive questions?
3. Try TF-IDF features instead of raw counts -- does classification improve?